

STM32H7x5/x7 dual-core microcontroller debugging

Introduction

The STM32H7x5/x7 dual-core microcontroller lines as described in [Table 1. Applicable products](#) (named STM32H7x5/x7 microcontrollers in this document) are based on the high-performance Arm® Cortex®-M7 and Cortex®-M4 32-bit RISC cores.

The STM32H7x5/x7 microcontrollers require a specific development approach so the application can take maximum advantage dual core architecture.

This application note provides guidelines for the debug of custom applications which run on the STM32H7x5/x7 microcontrollers using the software Toolchains: EWARM, MDK-ARM and SW4STM32.

The following subjects are addressed;

- An overview of the new IDE features which supports the dual core products.
- How to use EWARM, MDK-ARM and SW4STM32 with a dual-core device using the ST-LINK debug.

For more information on the STM32H7x5/x7 microcontrollers, refer to the following documents on www.st.com:

- STM32H745xI/G datasheet (DS12923)
- STM32H755xI datasheet (DS12919)
- *STM32F7 Series and STM32H7 Series Cortex®-M7 processor programming manual* (PM0253)

Table 1. Applicable products

Type	Product lines
Microcontrollers	STM32H745/755, STM32H747/757

1 General information

This document applies to the STM32H7x5/x7 dual-core Arm®-based microcontroller lines.

Note: Arm is a registered trademark of Arm Limited (or its subsidiaries) in the US and/or elsewhere.



2 Dual debug main features

A multicore processor is composed of either heterogeneous cores (which means different cores) or homogenous cores (identical cores).

The STM32H7x5/x7 dual-core microcontrollers use an heterogeneous core architecture. It is composed of an Arm® Cortex®-M7 core and an Arm® Cortex®-M4 core.

The two cores boot either separately or together. This is configurable using dedicated option bytes BCM7 and BCM4.

2.1 Access port

The STM32H7x5/x7 microcontrollers contain four access ports (AP) attached to the debug port (DP):

1. AP0: Cortex®-M7 access port (AHB-AP) gives access to the debug and trace features integrated in the Cortex®-M7 processor core via an AHB-Lite bus connected to the AHBD port of the processor.
2. AP1: D3 access port (AHB-AP) which allows access to the bus matrix in the D3 domain. The D3 domain memory and peripherals are be visible when the D1 and D2 domains are switched off. No Arm® CoreSight™ components are accessible via this port.
3. AP2: System access port (APB-AP) allows access to the debug and trace features on the APB debug system bus, that includes all components that are not part of any of the processor cores.
4. AP3: Cortex®-M4 access port (AHB-AP) allows access to the debug and trace features integrated in the Cortex®-M4 processor core via its internal AHB bus.

2.2 Cross trigger interface (CTI) and cross trigger matrix (CTM)

The cross trigger interfaces (CTI) and cross trigger matrix (CTM) together form the Arm® embedded cross trigger feature. There are three CTI components, one at system level, one dedicated to the Cortex®-M7 and one dedicated to the Cortex®-M4. The three CTIs are connected to each other via the CTM as illustrated in [Figure 1](#). The system-level and the Cortex®-M4 CTIs are accessible to the debugger via the system access port and associated APB-D.

The Cortex®-M7 CTI is physically integrated in the Cortex®-M7 core, and is accessible via the Cortex®-M7 access port and associated AHBD.

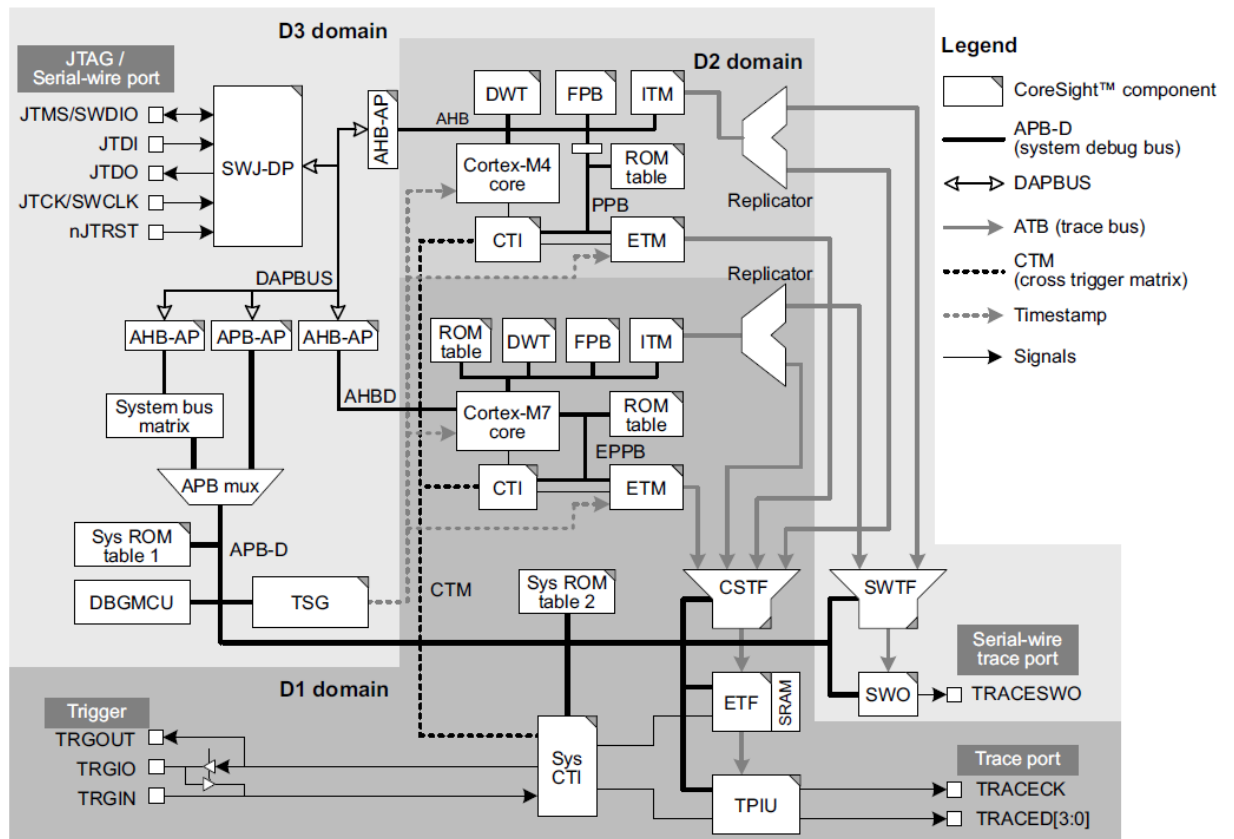
[illegible]

For more information on cross trigger input and output signals for each CTI, refer to the reference manual *STM32H745/755 and STM32H747/757 advanced Arm®-based 32-bit MCUs* (RM0399).

2.3 Debug power domain

The debug components are distributed across the power domains D1 , D2 and D3 as illustrated in Figure 2.

Figure 2. Debug power domain

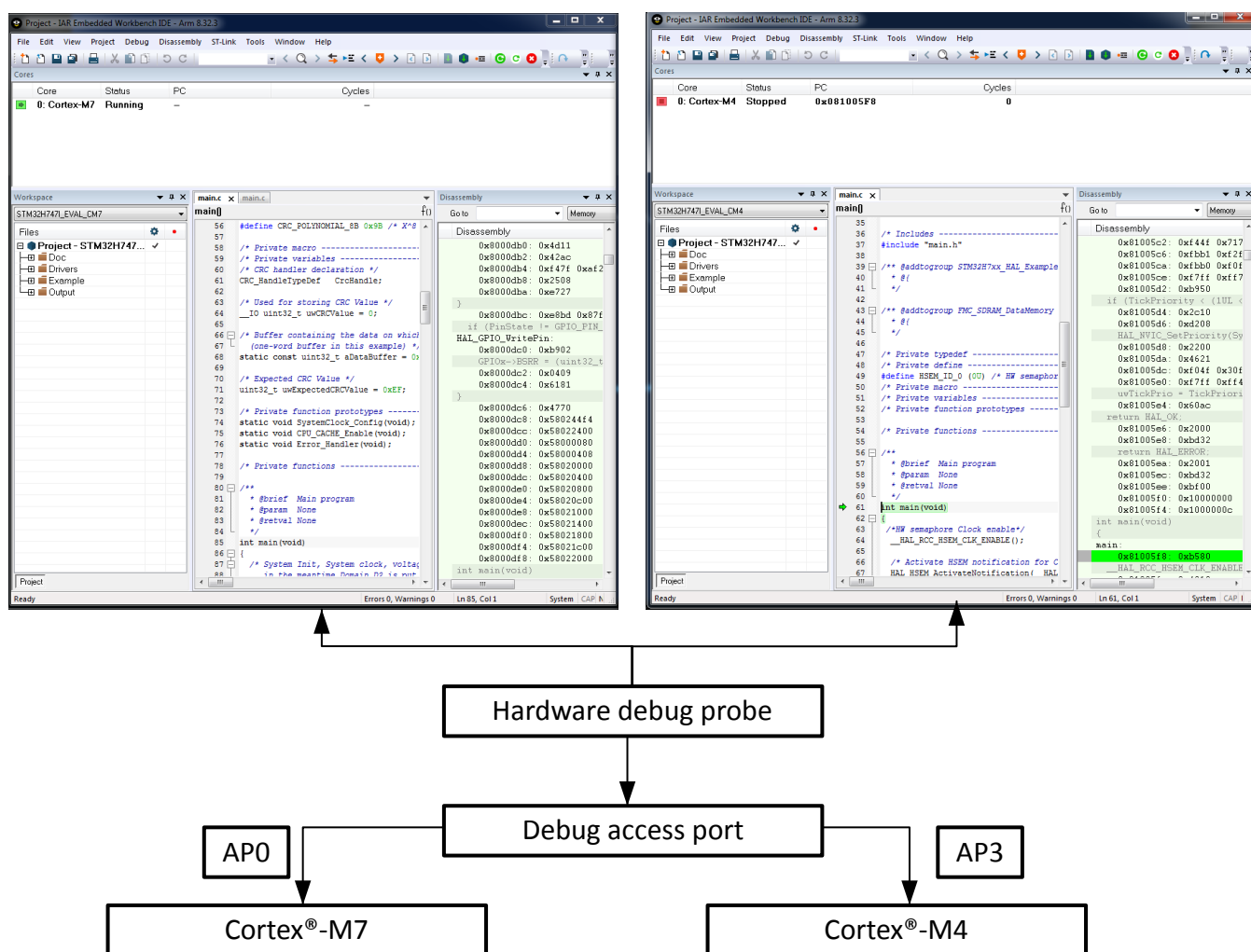


- D1 domain (highlighted in dark gray): contains the Cortex®-M7 core with the associated debug and trace components. It also contains the system trace components located on the APB-D which are common to both processors. This power domain therefore needs to be active whenever the Cortex®-M7 debug access is required, or when the trace functionality is active on either processor.
- D2 domain (highlighted in mid gray) only contains the debug and trace components dedicated to the Cortex®-M4. Removing this power domain does not impact the Cortex®-M7 debug and trace.
- D3 Domain (highlighted in light gray) must be active when the debugger is connected to it as it contains the serial wire JTAG debug Port (SWJDP). This ensures the debugger does not lose the connection with the system on chip (SoC) when one or both of the other power domains are switched off. In addition, it contains the time-stamp generator, the DBG_MCU and the serial wire trace features which allow the Cortex®-M7 core to be switched off while still allowing basic trace on the Cortex®-M4.

3 Debuggers (software and hardware)

Dual-core debugging allows the simultaneous debugging of both cores using a single hardware debug probe. The debug information for both cores is displayed either in a single IDE GUI or have an IDE GUI instance for each core. A representation of separate IDE GUI instances is illustrated in Figure 3.

Figure 3. Dual debugger



To ensure dual debug, a debugger must offer:

- access port selection capability
- simultaneous multiple core connection capability using the same debug probe
- visibility of all cores
- support of cross trigger Arm® component
- possibility to switch between access ports during the same debug session to visualize memory and peripherals statutes in the other domains.

The supported dual-core management features differ from a debugger to another.

In the STM32H7x5/x7 microcontrollers, the Flash memory is divided into two banks:

- CM7 uses bank1 from 0x0800 0000 to 0x080F FFFF
- CM4 uses bank2 from 0x0810 0000 to 0x081F FFFF.

Figure 4 illustrates the boot address options.

Figure 4. Boot address

Boot address option bytes

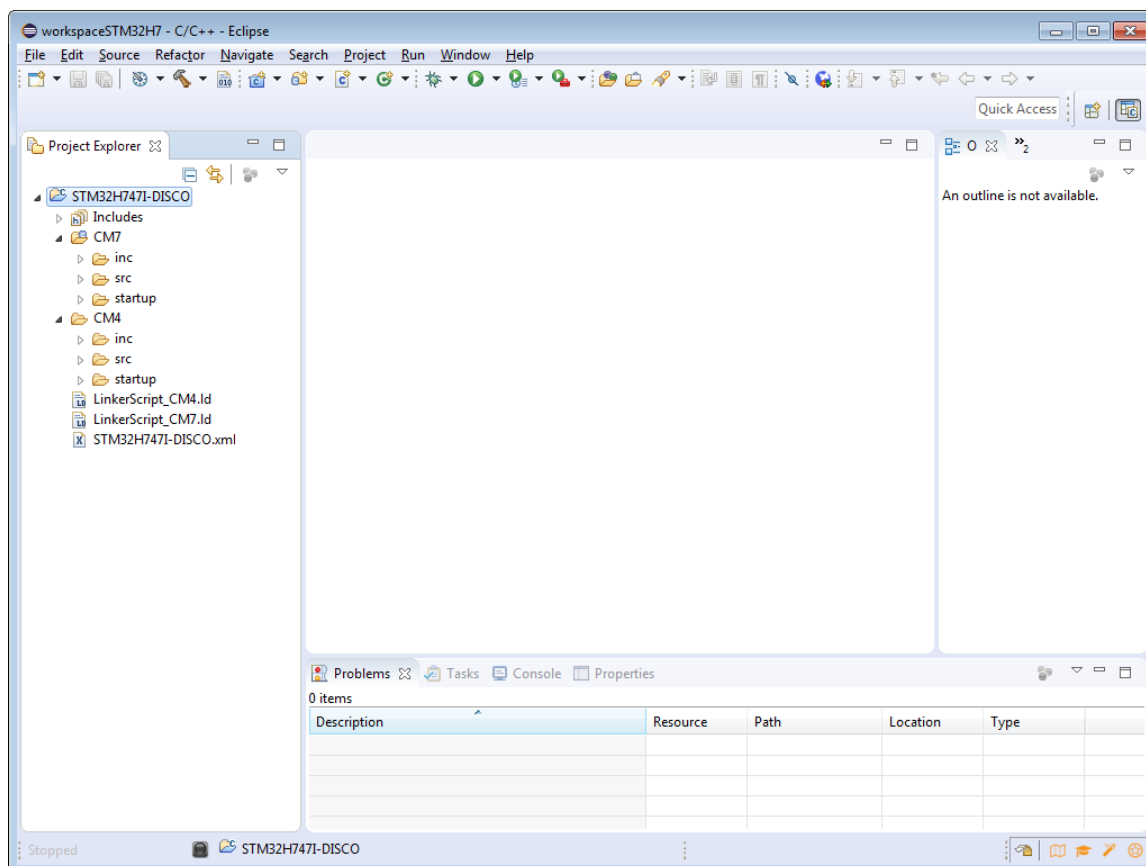
CM7_BOOT_ADD0 (H)	0x0800	Boot from (H)	0x08000000
CM7_BOOT_ADD1 (H)	0x1FF0	Boot from (H)	0x1FF00000
CM4_BOOT_ADD0 (H)	0x0810	Boot from (H)	0x08100000
CM4_BOOT_ADD1 (H)	0x1000	Boot from (H)	0x10000000

6 Using the STM32 system workbench

The latest version of system workbench for STM32 products (SW4STM32) is available to download from the official AC6 web site www.openstm32.org.

When creating a new project, both cores are included in a single project. A set of sources are dedicated to the CM4, another to the CM7 and where needed the user adds common sources built for both cores. The build generates two binaries, one for each core as shown in Figure 50.

Figure 50. Multicore project



6.1 Dual debugging on SW4STM32

System workbench for STM32 products offers the possibility to simultaneously debug two or more identical cores using either the symmetric multicore processing (SMP), or two cores with different architectures also known as asymmetric multicore processing (AMP). The debug is done either by using the same project for the multicore case (one board for X cores) or separate projects for different core configurations, each core/project using the shareable ST-Link to address each target.

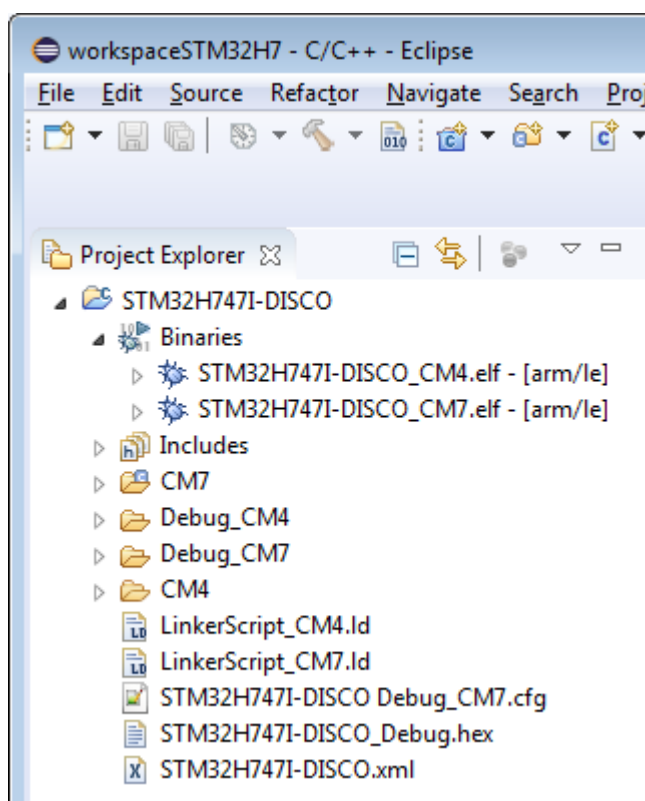
6.2 Steps for dual debug with ST-LINK V2/V3

This section provides step by step instructions to the STM32H7x5/x7 microcontrollers debug using SW4STM32 v2.9.0 and ST-LINK as a debug probe.

For this example, one project is created for both cores:

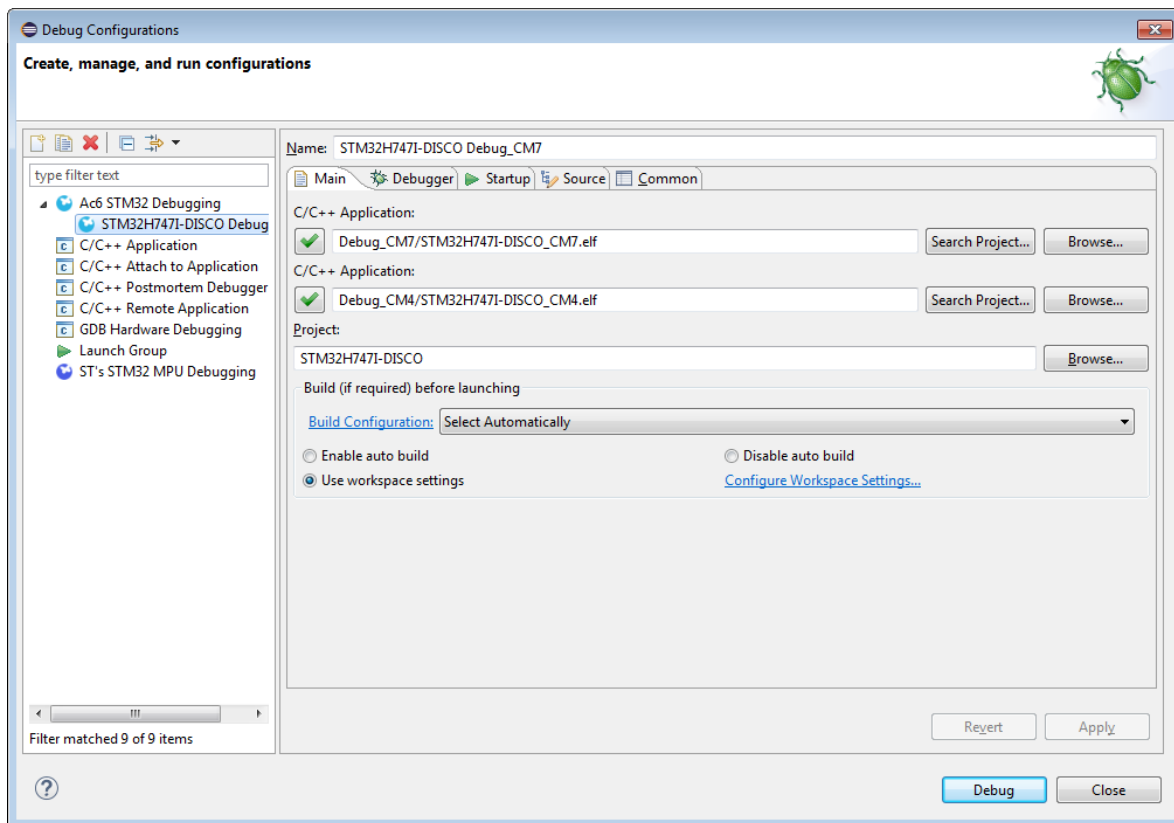
1. Open the STM32 system workbench and create one project for both cores.
2. Two binaries are generated by the compiler (Figure 51).

Figure 51. Multicore project: two binaries



3. Go to **Run** → **Debug Configurations** and double-click on Ac6 STM32 Debugging to create a new debug configuration (Figure 52):

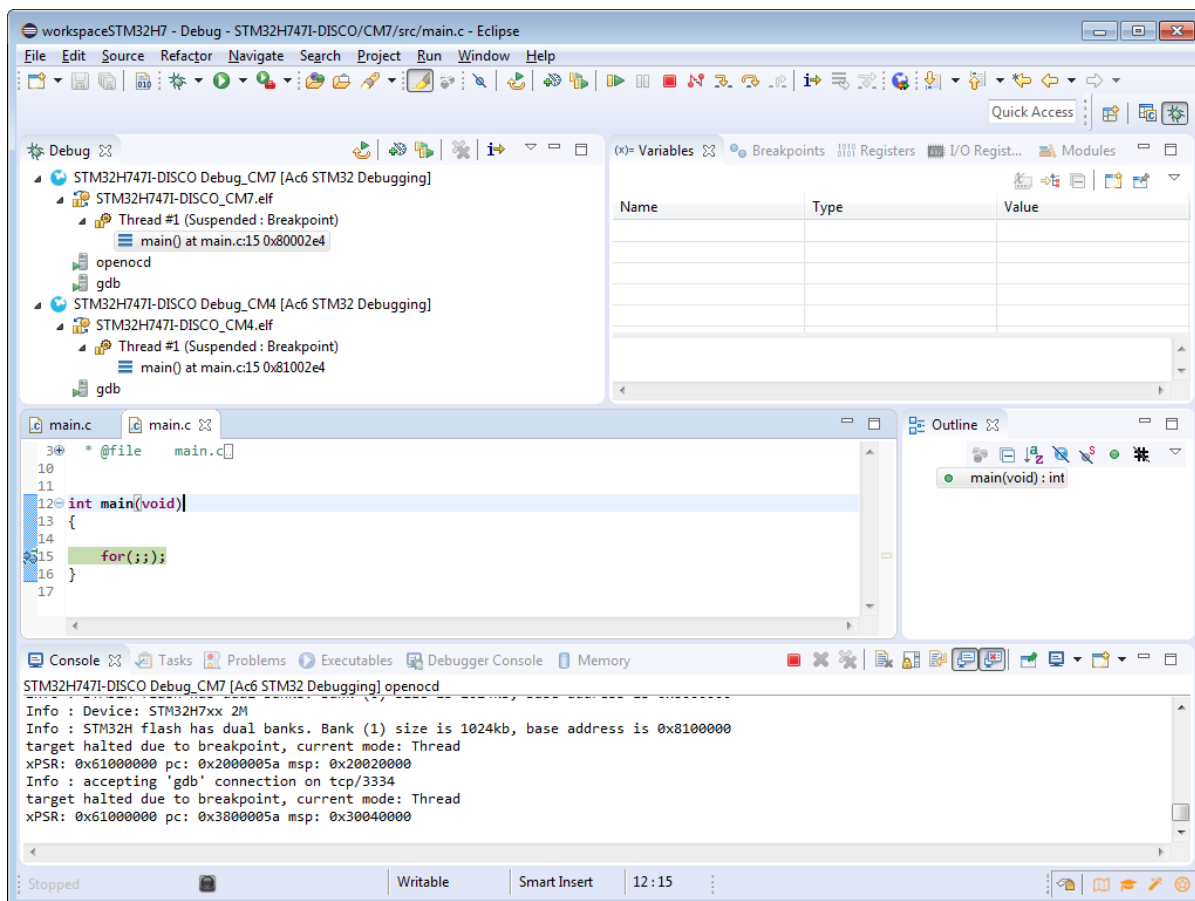
Figure 52. Multicore project: debug configuration



- a. Ensure both C/C++ Application fields are filled in.
- b. Click the  button to debug one or several cores at the same time.

4. Launch a debug session by clicking the Debug button in the Debug Configuration dialog box. In this example, both cores are launched at the same time and stop or break at the “main” instruction (Figure 53).

Figure 53. Multicore project: debug session



Break points are applied to both cores so they run either individually or simultaneously using the cross trigger interface (CTI). By default the CTI is not configured, therefore no core is able to break the other.

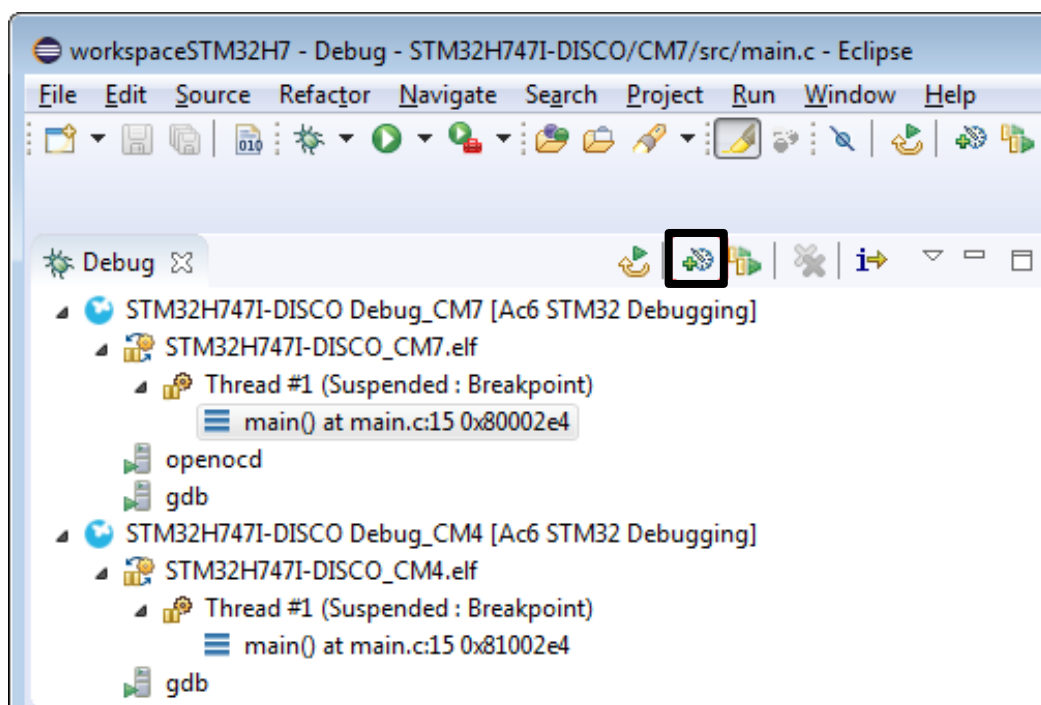
6.3 Embedded cross trigger (ECT) on SW4STM32

By default and as stated in [Section 6.2](#), the CTI is not activated.

To configure the CTI and stop both cores simultaneously, open the debug configuration that has already been created. In the debugger tab, go to the break cross-trigger matrix and select how the core breaks are configured, one breaks the other or both break at the same time. In the current configuration ([Figure 54](#)), both cores break at the “main” instruction which means when one core breaks the other breaks simultaneously.

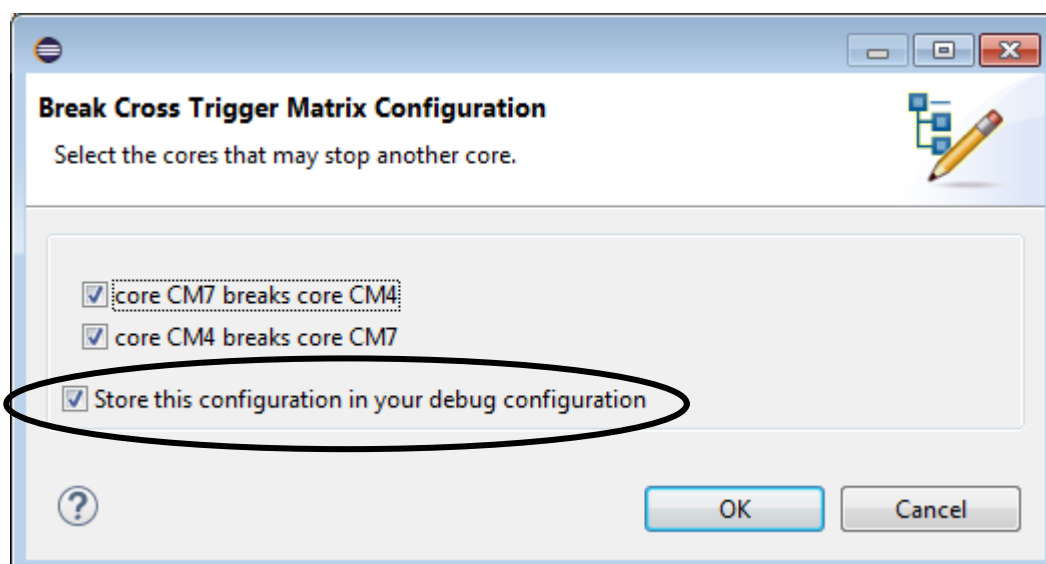
The CTI can be changed on the fly by clicking on Configure the BCTM button ([Figure 54](#)).

Figure 54. Multicore project: BCTM button



A new dialog box opens (Figure 55). It is possible to store the new BCTM configuration in the associated debug configuration.

Figure 55. Multicore project: BCTM dialog



Revision history

Table 2. Document revision history

Date	Version	Changes
01-Jul-2019	1	Initial release.
22-Aug-2024	2	Updated: - Section 5.3: Embedded cross trigger (ECT) on MDK-ARM - Section 5.2: Steps for dual debug with ST-LINK V2/V3 - Figure 12. ST-LINK → Multicore - Figure 24. Debugger → Multicore tab - Figure 32. Options → Debugger tab - Figure 37. Flash Download Tab - Figure 46. Debug session - Figure 47. Options for Target → Debug tab Added: - Figure 48. CTI internal connections - Figure 49. Debug session → Toolbox